

Лабораторная работа №13

Мартынова Милана Александровна

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	8
5. Выводы	11
Список литературы	12

Список иллюстраций

1	Первый скрипт	8
2	Второй скрипт	9
3	Третий скрипт	9
4	Четвертый скрипт	10
5	Запуск	10

Список таблиц

1. Цель работы

Освоить базовые принципы программирования в оболочке UNIX и научиться писать сложные сценарии (командные файлы), используя логические конструкции и циклы.

2. Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до $\square$ (например 1.tmp, 2.tmp, 3.tmp,4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

3. Теоретическое введение

Bash — это популярная командная оболочка UNIX, обеспечивающая интерактивную работу с командами и возможность написания скриптов для автоматизации. Её ключевые особенности: обработка простых и сложных команд (конвейеры, перенаправление, if/else, while), удобный ввод с автодополнением и историей, а также создание исполняемых скриптов с циклами и функциями. Работает в Unix/Linux и Windows (через WSL).

4. Выполнение лабораторной работы

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.(рис. 1)

```
mamartynova@mamartynova:~$ cat a.sh
#!/bin/bash

input_file=""
output_file=""
pattern=""
case_sensitive=""
show_line_numbers=""

while getopts "i:o:p:Cn" opt; do
  case $opt in
    i) input_file="$OPTARG" ;;
    o) output_file="$OPTARG" ;;
    p) pattern="$OPTARG" ;;
    C) case_sensitive="-i" ;;
    n) show_line_numbers="-n" ;;
    *) echo "Usage: $0 -i input -o output -p pattern [-C] [-n]"; exit 1 ;;
  esac
done

if [[ -z "$input_file" ]] || [[ -z "$pattern" ]]; then
  echo "Error: -i and -p are required"
  exit 1
fi

grep_cmd="grep $case_sensitive $show_line_numbers \"\$pattern\" \"\$input_file\""

if [[ -n "$output_file" ]]; then
  eval "$grep_cmd" > "$output_file"
  echo "Result saved to $output_file"
else
  eval "$grep_cmd"
fi
mamartynova@mamartynova:~$
```

Рис. 1: Первый скрипт

2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается

с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Команд- ный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.(рис. 2)

```
mamartynova@mamartynova:~$ cat b.sh
#!/bin/bash
./checknum
exit_code=$?

case $exit_code in
  0) echo "Сообщение оболочки: число > 0" ;;
  1) echo "Сообщение оболочки: число < 0" ;;
  2) echo "Сообщение оболочки: число = 0" ;;
  *) echo "Неизвестный код возврата" ;;
esac
mamartynova@mamartynova:~$
```

Рис. 2: Второй скрипт

3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до $\square$ (например 1.tmp, 2.tmp, 3.tmp,4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же ко- мандный файл должен уметь удалять все созданные им файлы (если они существуют).(рис. 3)

```
mamartynova@mamartynova:~$ cat c.sh
#!/bin/bash

action=$1
count=$2

if [[ "$action" == "create" ]]; then
  for ((i=1; i<=count; i++)); do
    touch "${i}.tmp"
    echo "Создан файл ${i}.tmp"
  done
elif [[ "$action" == "delete" ]]; then
  for ((i=1; i<=count; i++)); do
    rm -f "${i}.tmp"
    echo "Удалён файл ${i}.tmp"
  done
else
  echo "Использование: $0 {create|delete} N"
fi
mamartynova@mamartynova:~$
```

Рис. 3: Третий скрипт

4. Написать командный файл, который с помощью команды `tag` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`). (рис. 4)

```

mamartynova@mamartynova:~$ cat d.sh
#!/bin/bash

DIR=$1
ARCHIVE=$2

if [[ -z "$DIR" ]] || [[ -z "$ARCHIVE" ]]; then
    echo "Использование: $0 <директория> <архив.tar>"
    exit 1
fi

find "$DIR" -type f -mtime -7 -print > files_to_backup.txt

if [[ -s files_to_backup.txt ]]; then
    tar -cvf "$ARCHIVE" -T files_to_backup.txt
    echo "Архив $ARCHIVE создан (файлы изменённые за последние 7 дней)"
else
    echo "Нет файлов, изменённых менее 7 дней назад"
fi

rm -f files_to_backup.txt
mamartynova@mamartynova:~$

```

Рис. 4: Четвертый скрипт

Делаю файлы исполняемыми и запускаю. (рис. 5)

```

mamartynova@mamartynova:~/home/mamartynova
mamartynova@mamartynova:~$ chmod +x c.sh
mamartynova@mamartynova:~$ ./c.sh create 5
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
Создан файл 4.tmp
Создан файл 5.tmp
mamartynova@mamartynova:~$ ls -la *.tmp
-rw-r--r-- 1 mamartynova mamartynova 0 мая  7 16:24 1.tmp
-rw-r--r-- 1 mamartynova mamartynova 0 мая  7 16:24 2.tmp
-rw-r--r-- 1 mamartynova mamartynova 0 мая  7 16:24 3.tmp
-rw-r--r-- 1 mamartynova mamartynova 0 мая  7 16:24 4.tmp
-rw-r--r-- 1 mamartynova mamartynova 0 мая  7 16:24 5.tmp
mamartynova@mamartynova:~$ ./c.sh delete 5
Удалён файл 1.tmp
Удалён файл 2.tmp
Удалён файл 3.tmp
Удалён файл 4.tmp
Удалён файл 5.tmp
mamartynova@mamartynova:~$ ls *.tmp
ls: невозможно получить доступ к '*.tmp': Нет такого файла или каталога
mamartynova@mamartynova:~$ mkdir testdir
mamartynova@mamartynova:~$ cd testdir
mamartynova@mamartynova:~/testdir$ echo "old" > old.txt
mamartynova@mamartynova:~/testdir$ touch -t 202001011200 old.txt
mamartynova@mamartynova:~/testdir$ echo "new" > new.txt
mamartynova@mamartynova:~/testdir$ cd ..
mamartynova@mamartynova:~$ chmod +x d.sh
mamartynova@mamartynova:~$ ./d.sh testdir backup.tar
testdir/new.txt
Архив backup.tar создан (файлы изменённые за последние 7 дней)
mamartynova@mamartynova:~$ tar -tvf backup.tar
-rw-r--r-- mamartynova/mamartynova 4 2026-05-07 16:26 testdir/new.txt
mamartynova@mamartynova:~$ cat c.sh

```

Рис. 5: Запуск

5. Выводы

Мы освоили базу программирования в оболочке UNIX и приобрели навыки написания усложнённых командных скриптов, применяя логические конструкции и циклы.

Список литературы